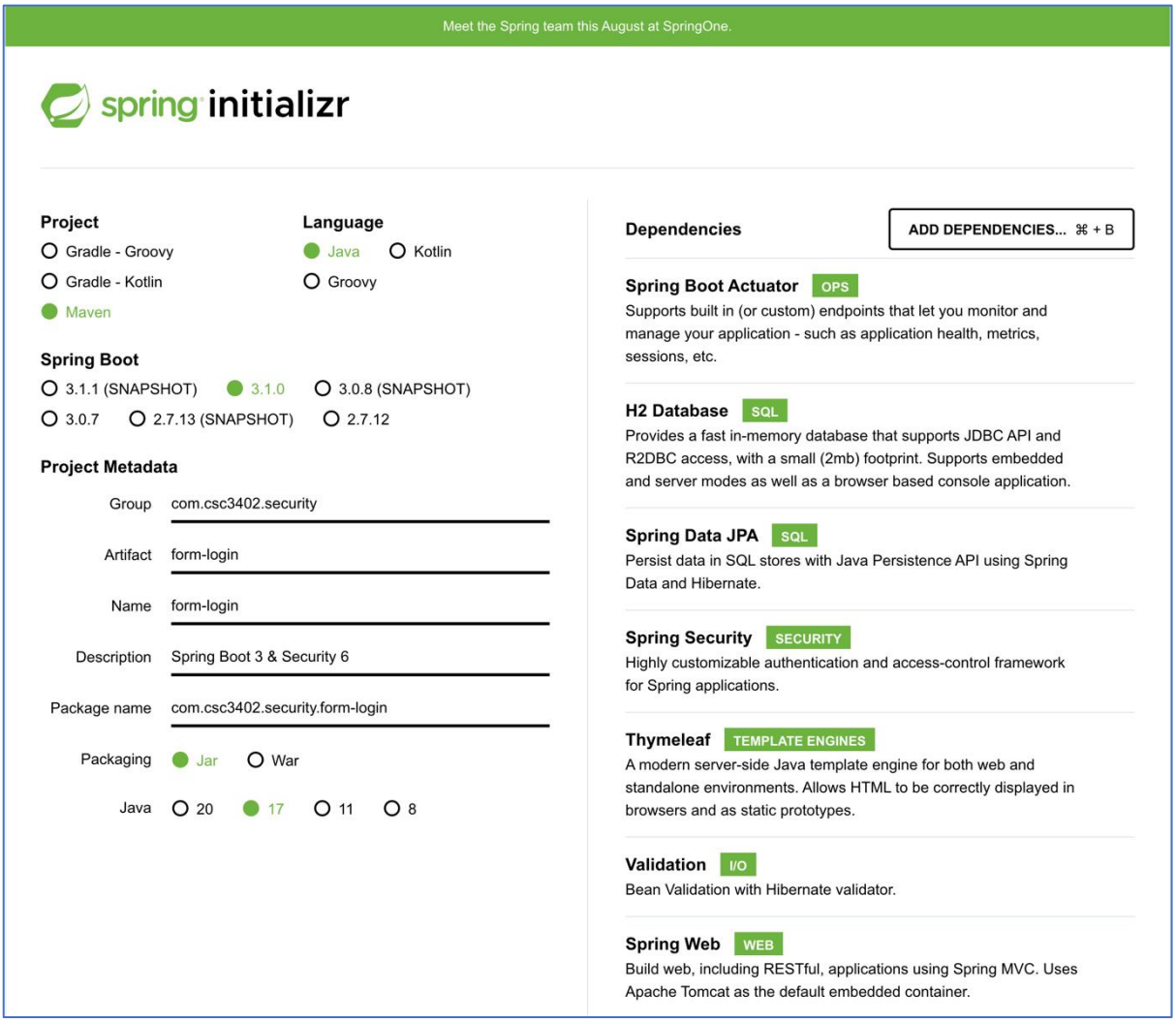
Lab 12

Spring Boot login/Logout (Spring Security + H2 + Thymeleaf)

In this lab we shall implement "login/logout" and "authentication/authorization" in a "Spring Boot 3" application.

To demonstrate the functionality, a new "Spring Boot" application is created with the help of spring initializr.



In order to save/get user details (name, email or username, roles) the application shall use the H2 database. The application also uses Thymeleaf as a server-side template engine. We shall create a custom "registration" and "login" flow based on "thymeleaf (html)" forms.

Dependencies

pom.xml

The "spring-boot-starter-security" combines Spring Security dependencies, and the "spring-boot-starter-web" enables "Tomcat" as a default embedded server.

Like wise, "spring-boot-starter-data-jpa" provides the "data-jpa" dependency to efficiently connect Spring applications with relational databases.

Below shows required dependencies:

```
<dependencies>
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-actuator</artifactId>
    </dependency>
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-data-jpa</artifactId>
    </dependency>
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-security</artifactId>
    </dependency>
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-thymeleaf</artifactId>
    </dependency>
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-validation</artifactId>
    </dependency>
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-web</artifactId>
    </dependency>
    <dependency>
        <groupId>org.thymeleaf.extras</groupId>
        <artifactId>thymeleaf-extras-springsecurity6</artifactId>
    </dependency>

    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-test</artifactId>
        <scope>test</scope>
    </dependency>
    <dependency>
        <groupId>org.springframework.security</groupId>
        <artifactId>spring-security-test</artifactId>
        <scope>test</scope>
    </dependency>
    <dependency>
        <groupId>com.h2database</groupId>
        <artifactId>h2</artifactId>
        <scope>runtime</scope>
    </dependency>
</dependencies>
```

Model Package

Create two classes named as “User” and “Role” and annotate as entities (@Entity) in a package called “model”.

Many-to-many relationship between User and Role.

```java
@Entity
@Table(name = "users")
public class User {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false)
    private String name;

    @Column(nullable = false, unique = true)
    private String email;

    @Column(nullable = false)
    private String password;

    @ManyToMany(fetch = FetchType.EAGER, cascade = CascadeType.ALL)
    @JoinTable(
            name = "users_roles",
            joinColumns = {@JoinColumn(name = "user_id", referencedColumnName = "id")},
            inverseJoinColumns = {@JoinColumn(name = "role_id", referencedColumnName = "id")}
    )
    private List<Role> roles = new ArrayList<>();

    public User() {
    }

    public User(String name, String email, String password, List<Role> roles) {
        this.name = name;
        this.email = email;
        this.password = password;
        this.roles = roles;
    }
```

Source path: src/main/java/com/csc3402/security/formlogin/model/User.java

Note: Remember to create

1. Getters & Setters,
2. All-arguments & non-arguments constructor,
3. toString

The "User" and "Role" are plain old models representing a Spring Security user and associated role in the application and database.

```java
@Entity
@Table(name = "roles")
public class Role {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false, unique = true)
    private String name;

    @ManyToMany(mappedBy = "roles")
    private List<User> users = new ArrayList<>();

    public Role(String name) { this.name = name; }

    public Role() {
    }

    public Role(String name, List<User> users) {
        this.name = name;
        this.users = users;
    }
```

source path: src/main/java/com/csc3402/security/formlogin/model/Role.java

Note: Remember to create

1. Getters & Setters,
2. All-arguments & non-arguments constructor,
3. toString

The "Role" class has a list of users, which means a user can have multiple roles, and a role can be associated with a number of users.

DTO Package

DAO (Data Access Object) and DTO (Data Transfer Object) patterns are used in Object Relational Mapping. DAO acts as a bridge between the database and the application. DTO acts as a data store from where the data is received and transferred to different layers i.e., to the DAO application.

Create a package called 'dto' and a class called "UserDto"

```java
import jakarta.validation.constraints.Email;
import jakarta.validation.constraints.NotEmpty;

public class UserDto {

    private Long id;

    @NotEmpty(message = "Please enter valid name.")
    private String name;

    @NotEmpty(message = "Please enter valid email.")
    @Email
    private String email;

    @NotEmpty(message = "Please enter valid password.")
    private String password;

    public UserDto() {
    }

    public UserDto(String name, String email, String password) {
        this.name = name;
        this.email = email;
        this.password = password;
    }
```

Source path: src/main/java/com/csc3402/security/formlogin/dto/UserDto.java

Note: Remember to create

1. Getters & Setters,
2. All-arguments & non-arguments constructor,
3. toString
4.

The "UserDto" class is a helper class to be used in "Html form to Controller communication" when a new user is registered in the system.

Repository Package

Create two repositories called “UserRepository” and “RoleRepository” in repository package The "UserRepository" and "RoleRepository" are using Spring Data JPA to significantly improve the implementation of data access layers by reducing the effort to the amount that's actually needed.

```java
package com.csc3402.security.formlogin.repository;

import com.csc3402.security.formlogin.model.User;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.stereotype.Repository;

@Repository
public interface UserRepository extends JpaRepository<User, Long> {
    User findByEmail(String email);
}

```

Source path: src/main/java/com/csc3402/security/formlogin/repository/UserRepository.java

The "findByName" method gets the "Role" object for the associated "role" name.

```java
package com.csc3402.security.formlogin.repository;

import com.csc3402.security.formlogin.model.Role;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.stereotype.Repository;

@Repository
public interface RoleRepository extends JpaRepository<Role, Long> {
    Role findByName(String name);
}

```

Source path: src/main/java/com/csc3402/security/formlogin/repository/RoleRepository.java

Constants (util package)

The "TbConstants.java" maintains constants used throughout the application; this helps make the application code more maintainable.

Create a package called “util” and a java class called “TbConstants” as follows :-

TbConstants.java | user.html | application.properties

```
1  package com.csc3402.security.formlogin.util;
2
3  public class TbConstants {
4      public static interface Roles {
5          String USER = "ROLE_USER";
6          String ADMIN = "ROLE_ADMIN";
7      }
8  }
9
```

Service Package

Next, create a package call “service” and an interface and class as follows:-

1. UserService.java - Interface
2. UserServiceImpl.java - class

"UserService.java" and "UserServiceImpl.java" interface represent an intermediate service layer between data/repository and controller.

UserService.java

```
1  package com.csc3402.security.formlogin.service;
2
3  import com.csc3402.security.formlogin.dto.UserDto;
4  import com.csc3402.security.formlogin.model.User;
5
6  public interface UserService {
7
8      void saveUser(UserDto userDto);
9      User findUserByEmail(String email);
10 }
11
```

Source path: src/main/java/com/csc3402/security/formlogin/service/UserService.java

The "saveUser()" method converts "UserDto" to "User" and saves it in the database (H2).
Note: UserServiceImpl is annotated with **@Service** annotation

```
14
15  @Service
16  public class UserServiceImpl implements UserService{
17      @Autowired
18      private UserRepository userRepository;
19
20      @Autowired
21      private RoleRepository roleRepository;
22
23      @Autowired
24      private PasswordEncoder passwordEncoder;
25      @Override
26
27      public void saveUser(UserDto userDto) {
28          Role role = roleRepository.findByName(TbConstants.Roles.USER);
29
30          if (role == null)
31              role = roleRepository.save(new Role(TbConstants.Roles.USER));
32
33          User user = new User(userDto.getName(), userDto.getEmail(), passwordEncoder.encode(userDto.getPassword()),
34                  Arrays.asList(role));
35          userRepository.save(user);
36      }
37
38      @Override
39      public User findUserByEmail(String email) { return userRepository.findByEmail(email); }
42  }
43
```

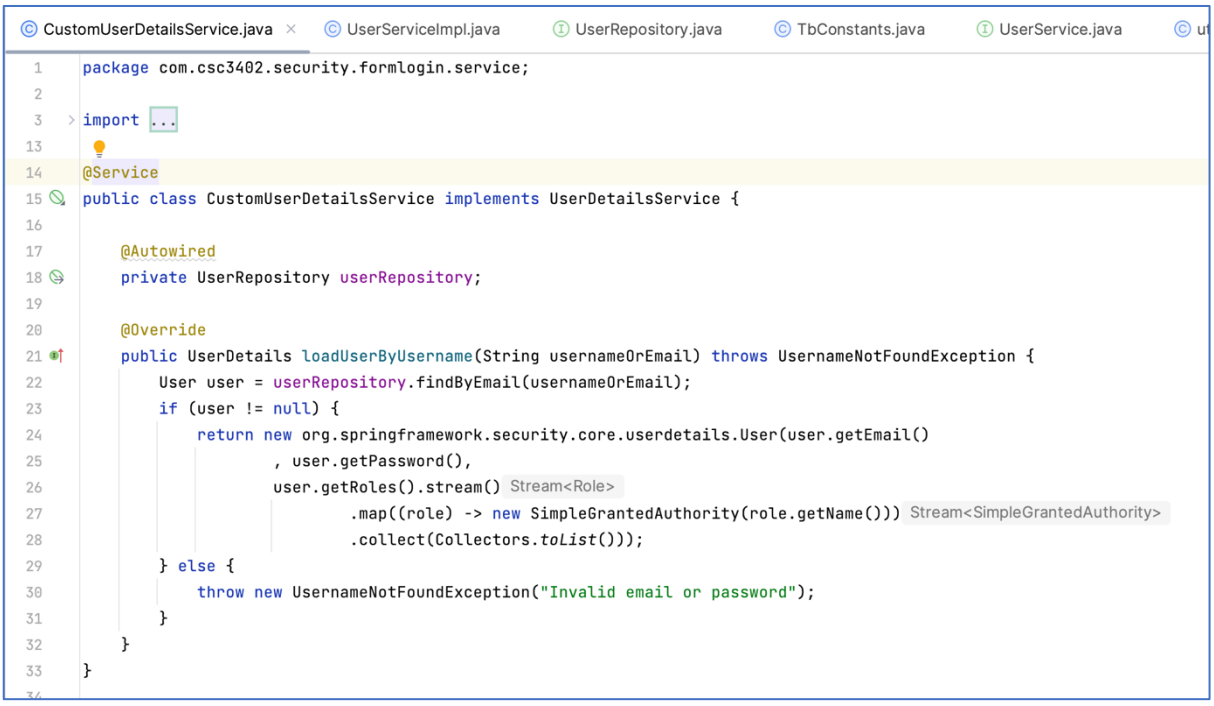
Source path: src/main/java/com/csc3402/security/formlogin/service/UserServiceImpl.java

Below, **"CustomUserDetailsService"** is a custom implementation of **"UserDetailsService"** and is used by Spring to get user details from the database during a login flow.

What is the use of **UserDetailsService**?

The UserDetailsService interface from Spring Security is used to retrieve user-related data. It has one method named loadUserByUsername() which can be overridden to customize the process of finding the user. It is used by the DaoAuthenticationProvider to load details about the user during authentication.

Note: CustomUserDetailsService is annotated with **@Service** annotation.



Source path: src/main/java/com/csc3402/security/formlogin/service/CustomUserDetailsService.java

Controller Package

This "LoginController" hosts "request-mappings" for "Login" and "Registration" related UI (HTML + Thymeleaf)

LoginController.java | CustomUserDetailsService.java | UserController.java | UserServiceImpl.java

```java
import ...

@Controller
public class LoginController {
    @Autowired
    private UserService userService;

    @RequestMapping("/login")
    public String loginForm() { return "login"; }

    @GetMapping("/registration")
    public String registrationForm(Model model) {
        UserDto user = new UserDto();
        model.addAttribute("user", user);
        return "registration";
    }

    @PostMapping("/registration")
    public String registration(
            @Valid @ModelAttribute("user") UserDto userDto,
            BindingResult result,
            Model model) {
        User existingUser = userService.findUserByEmail(userDto.getEmail());

        if (existingUser != null)
            result.rejectValue("email", null,
                    "User already registered !!!");

        if (result.hasErrors()) {
            model.addAttribute("user", userDto);
            return "/registration";
        }

        userService.saveUser(userDto);
        return "redirect:/registration?success";
    }

    @GetMapping("/logout")
    public String userLogout() { return "redirect:/login?logout"; }
}
```

Source path: src/main/java/com/csc3402/security/formlogin/controller/LoginController.java

This "UserController" hosts a single "request-mappings" for "user-details" related UI (HTML + Thymeleaf).

Configuration Package (Security Configuration)

The "SecurityFilterChain" defines a filter chain that is capable of being matched against an HttpServletRequest.

As mentioned in the configuration below, the urls matching with "/user/**" are accessible by users having either the "USER" or "ADMIN" role, whereas the urls matching with "/admin/**" are only accessible by users having the "ADMIN" role.

On the other hand, urls beginning with "/registration/**" and "/login/**" are accessible by both authenticated and non-authenticated users, or accessible by all.

Create a package called “config” and a java class called “SpringSecurity”. Code as follows:-

SpringSecurity.java | UserController.java | LoginController.java | CustomUserDetailsService.java | UserServiceImpl.java

```java
package com.csc3402.security.formlogin.config;

import ...

@Configuration
@EnableWebSecurity
public class SpringSecurity {

    @Bean
    public static PasswordEncoder passwordEncoder() { return new BCryptPasswordEncoder(); }

    @Bean
    public SecurityFilterChain securityFilterChain(HttpSecurity http) throws Exception {
        http
                .authorizeHttpRequests((requests) -> requests
                        .requestMatchers(AntPathRequestMatcher.antMatcher("/h2-console/**")).permitAll()
                        .requestMatchers("/registration/**").permitAll()
                        .requestMatchers("/login/**").permitAll()
                        .requestMatchers("/user/**").hasAnyRole("USER", "ADMIN")
                        .requestMatchers("/admin/**").hasAnyAuthority("ADMIN")
                        .anyRequest().authenticated()
                )
                .formLogin((form) -> form
                        .loginPage("/login")
                        .loginProcessingUrl("/login")
                        .defaultSuccessUrl("/user/")
                        .permitAll()
                )
                .logout((logout) -> logout.permitAll())
                .exceptionHandling((exceptionHandling) -> exceptionHandling.accessDeniedPage("/access-denied"))
                .csrf((csrf) -> csrf.ignoringRequestMatchers(AntPathRequestMatcher.antMatcher("/h2-console/**")))
                .headers((headers) -> headers.disable());
        return http.build();
    }
}
```

Source path: src/main/java/com/csc3402/security/formlogin/config/SpringSecurity.java

Application

This is the simple "spring boot" "main" class; all the application execution starts from here. The only interesting thing to note is the exclusion of "SecurityAutoConfiguration.class" and "ManagementWebSecurityAutoConfiguration.class". This is required to disable security auto-configuration so that we can have our own settings, like a custom login form.

FormLoginApplication.java | pom.xml (form-login) | UserController.java | LoginController.java | CustomUserDetailsService.java

```java
package com.csc3402.security.formlogin;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.actuate.autoconfigure.security.servlet.ManagementWebSecurityAutoConfiguration;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.boot.autoconfigure.security.servlet.SecurityAutoConfiguration;

@SpringBootApplication(exclude = {SecurityAutoConfiguration.class, ManagementWebSecurityAutoConfiguration.class})
public class FormLoginApplication {

    public static void main(String[] args) { SpringApplication.run(FormLoginApplication.class, args); }

}
```

Source path: src/main/java/com/csc3402/security/formlogin/FormLoginApplication java

HTML

The "**registration.html**" is a simple ".html" file with "thymeleaf" template capabilities. This page contains a server-side validation-enabled "registration" form.

HTML code: registration.html

```
<!doctype html>
<html lang="en" xmlns:th="http://www.thymeleaf.org">
<head>
  <meta charset="utf-8">
  <meta content="width=device-width, initial-scale=1" name="viewport">
  <link crossorigin="anonymous"
href="https://cdn.jsdelivr.net/npm/bootstrap@5.0.2/dist/css/bootstrap.min.css"
        integrity="sha384-EVSTQN3/azprG1Anm3QDgpJLIm9Nao0Yz1ztcQTwFspd3yD65VohhpuuCOmLASjC"
rel="stylesheet">

  <title>Registration</title>
</head>
<body>
<div class="container">
  <div class="row mt-5">
    <div class="col-6 mx-auto">
      <form
              method="post"
              role="form"
              th:action="@{/registration}"
              th:object="${user}"
      >
        <h1>Registration Form</h1>
        <div th:if="${param.success}">
          <div class="alert alert-info">
            Registration successful !!!
          </div>
        </div>
        <div class="mb-3">
          <label class="form-label" for="name">Name</label>
          <input class="form-control" id="name" name="name" type="text">
          <p class="text-danger" th:errors="*{name}"
             th:if="${#fields.hasErrors('name')}"></p>
        </div>
        <div class="mb-3">
          <label class="form-label" for="email">Email</label>
          <input class="form-control" id="email" name="email" type="email">
          <p class="text-danger" th:errors="*{email}"
             th:if="${#fields.hasErrors('email')}"></p>
        </div>
        <div class="mb-3">
          <label class="col-sm-2 col-form-label" for="password">Password</label>
          <input class="form-control" id="password" name="password" type="password">
          <p class="text-danger" th:errors="*{password}"
             th:if="${#fields.hasErrors('password')}"></p>
        </div>
        <div class="mb-3">
          <button class="btn btn-primary mb-3" type="submit">Register</button>
          or <a class="link-primary" th:href="@{/login}">Login</a>
        </div>
      </form>
    </div>
  </div>
</div>

<script crossorigin="anonymous"
        integrity="sha384-MrcW6ZMFYlzcLA8Nl+NtUVF0sA7MsXsP1UyJoMp4YLEuNSfAP+JcXn/tWtIaxVXM"

src="https://cdn.jsdelivr.net/npm/bootstrap@5.0.2/dist/js/bootstrap.bundle.min.js"></script
>

</body>
</html>
```

Source path: src/main/resources/templates/registration.html

By default spring security accepts login request at "/login" and credentials in the form of "username" and "password". Custom design user login form.

HTML code:login.html

```
<!doctype html>
<html lang="en" xmlns:th="http://www.thymeleaf.org">
<head>
  <meta charset="utf-8">
  <meta content="width=device-width, initial-scale=1" name="viewport">
  <link crossorigin="anonymous"
href="https://cdn.jsdelivr.net/npm/bootstrap@5.0.2/dist/css/bootstrap.min.css"
        integrity="sha384-EVSTQN3/azprG1Anm3QDgpJLIm9Nao0Yz1ztcQTwFspd3yD65VohhpuuCOmLASjC"
rel="stylesheet">

  <title>Login</title>
</head>
<body>
<div class="container">
  <div class="row mt-5">
    <div class="col-6 mx-auto">
      <h1>Login Form</h1>
      <div th:if="${param.error}">
        <div class="alert alert-danger">Invalid Email or Password !!!</div>
      </div>
      <div th:if="${param.logout}">
        <div class="alert alert-success">You have been logged out !!!</div>
      </div>
      <form
              class="form-horizontal"
              method="post"
              role="form"
              th:action="@{/login}"
      >
        <div class="mb-3">
          <label class="form-label" for="password">Email</label>
          <input class="form-control" id="username" name="username" placeholder="Enter
email address"
                 type="email">
        </div>
        <div class="mb-3">
          <label class="col-sm-2 col-form-label" for="password">Password</label>
          <input class="form-control" id="password" name="password" placeholder="Enter
password"
                 type="password">
        </div>
        <div class="mb-3">
          <button class="btn btn-primary mb-3" type="submit">Login</button>
          or <a class="link-primary" th:href="@{/registration}">Register</a>
        </div>
      </form>
    </div>
  </div>
</div>

<script crossorigin="anonymous"
        integrity="sha384-MrcW6ZMFYlzcLA8Nl+NtUVF0sA7MsXsP1UyJoMp4YLEuNSfAP+JcXn/tWtIaxVXM"

src="https://cdn.jsdelivr.net/npm/bootstrap@5.0.2/dist/js/bootstrap.bundle.min.js"></script
>

</body>
</html>
```

Source path: src/main/resources/templates/login.html

The "thymeleaf" template engine enables us to use "sec:authentication" tags to display security-related information in HTML.

HTML code: user.html

```
<!doctype html>
<html lang="en" xmlns:sec="http://www.thymeleaf.org/extras/spring-security"
      xmlns:th="http://www.thymeleaf.org">
<head>
  <meta charset="utf-8">
  <meta content="width=device-width, initial-scale=1" name="viewport">
  <link crossorigin="anonymous"
href="https://cdn.jsdelivr.net/npm/bootstrap@5.0.2/dist/css/bootstrap.min.css"
        integrity="sha384-EVSTQN3/azprG1Anm3QDgpJLIm9Nao0Yz1ztcQTwFspd3yD65VohhpuuCOmLASjC"
rel="stylesheet">

  <title>User</title>
</head>
<body>
<div class="container">
  <div class="row mt-5">
    <div class="col-6 mx-auto">
      <h1>User Details</h1>
      <div class="mt-2">Username: <span sec:authentication="principal.username"/></div>
      <div class="mt-2">   Roles: <span sec:authentication="principal.authorities"/></div>

      <div class="mt-5"><a class="link-primary" th:href="@{/logout}">Logout</a></div>
    </div>
  </div>
</div>


<script crossorigin="anonymous"
        integrity="sha384-MrcW6ZMFYlzcLA8Nl+NtUVF0sA7MsXsP1UyJoMp4YLEuNSfAP+JcXn/tWtIaxVXM"

src="https://cdn.jsdelivr.net/npm/bootstrap@5.0.2/dist/js/bootstrap.bundle.min.js"></script
>

</body>
</html>
```

Source path: src/main/resources/templates/user.html

Application Properties

This "**application.properties**" file contains "db credentials," "hibernate" settings configurations.

```
spring.datasource.url=jdbc:h2:mem:testdb
spring.datasource.driver-class-name=org.h2.Driver
spring.datasource.username=sa
spring.datasource.password=
spring.jpa.database-platform=org.hibernate.dialect.H2Dialect
# In order to get import_backup.sql populate data into H2 table
spring.sql.init.mode=always
# For Spring Boot 2.4+ add spring.jpa.defer-datasource-initialization=true in application.properties
spring.jpa.defer-datasource-initialization=true
# H2 Database Setting
spring.h2.console.enabled=true
###
#   Hibernate Settings
#   ddl.auto = none | validate | update | create | create-drop
spring.jpa.hibernate.ddl-auto = create-drop
spring.jpa.properties.hibernate.show_sql=true
spring.jpa.properties.hibernate.use_sql_comments=false
spring.jpa.properties.hibernate.format_sql=true
```

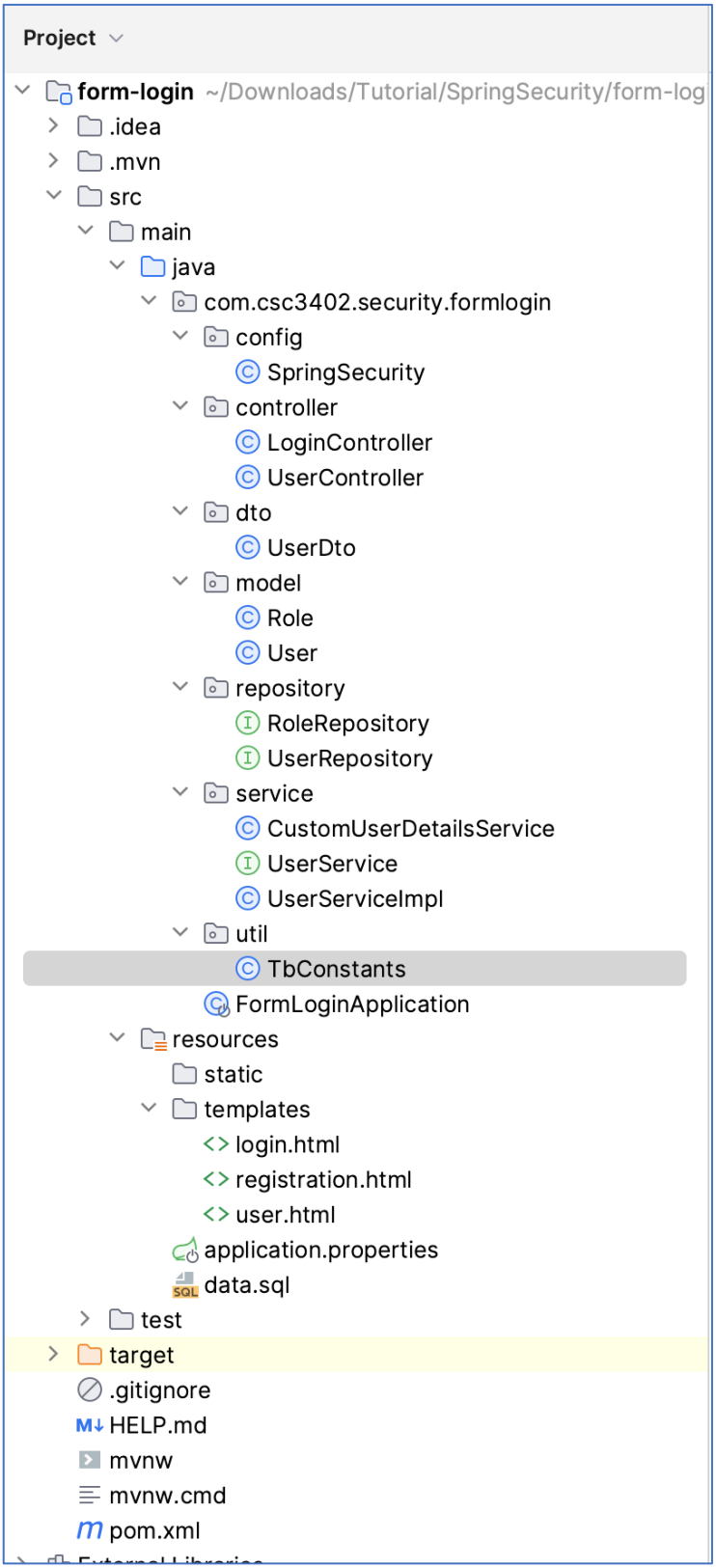
File name: data.sql

```
INSERT INTO users(email, name, password) VALUES
   ('admin@gmail.com','admin','$2a$12$JGriZzgFwZNEeuIzFcocjug9wb0/G0EJ1nco27FZoCvVLmfpfiiWe');
INSERT INTO users(email, name, password) VALUES
   ('test@gmail.com','John','$2a$12$F2QRPx07EsQTswGmyxB4sOw7RLCnIDqux/LWhO5vfegs42OS2LE.C');
INSERT INTO users(email, name, password) VALUES
   ('abc@gmail.com','Alex','$2a$12$aRRjmnRA5MkVg3M.xSc1G.jQkkvfx73WJhIvI.77IMkJyZpTT5dvC');
INSERT INTO users(email, name, password) VALUES
   ('def@gmail.com','Boss','$2a$12$MBeLkc4HGARn/pOnBl1HSugtEBvL5EwRvQ4EzqAykvt4hUogKl/Zy');

INSERT INTO roles(name) VALUES ('ROLE_ADMIN');
INSERT INTO roles(name) VALUES ('ROLE_USER');

INSERT INTO users_roles(role_id, user_id) VALUES (1,1);
INSERT INTO users_roles(role_id, user_id) VALUES (2,2);
INSERT INTO users_roles(role_id, user_id) VALUES (2,3);
INSERT INTO users_roles(role_id, user_id) VALUES (2,4);
INSERT INTO users_roles(role_id, user_id) VALUES (1,4);
```

Program Structure

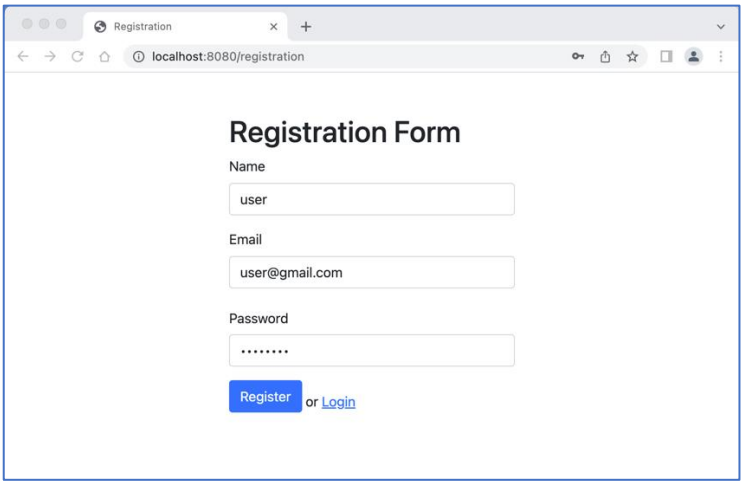


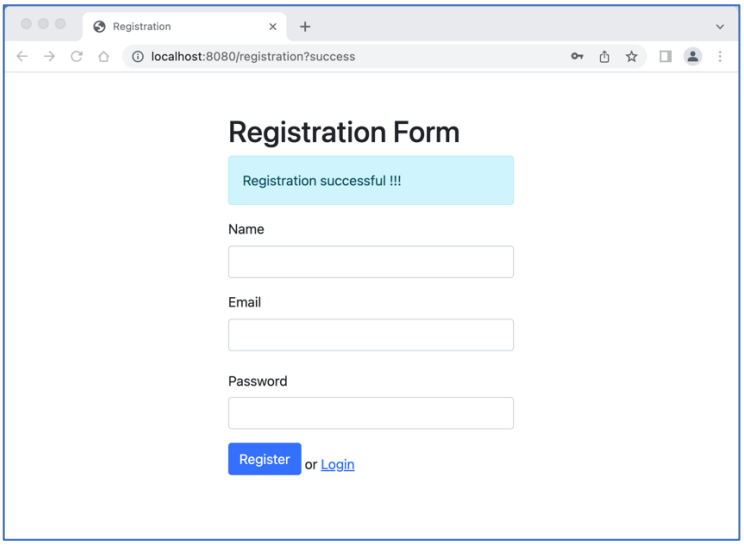
Testing

1) Registration

Let's run the application and start testing the functionality by registering a new user at : http://localhost:8080/registration.

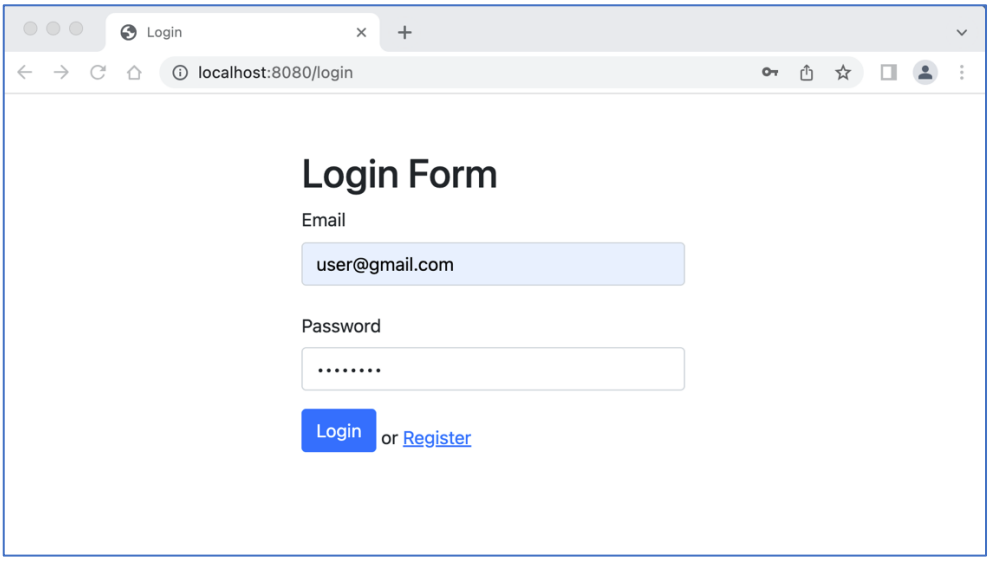
Notes: Currently all uploaded users (data.sql) are assigned password as “password”, for testing purpose.





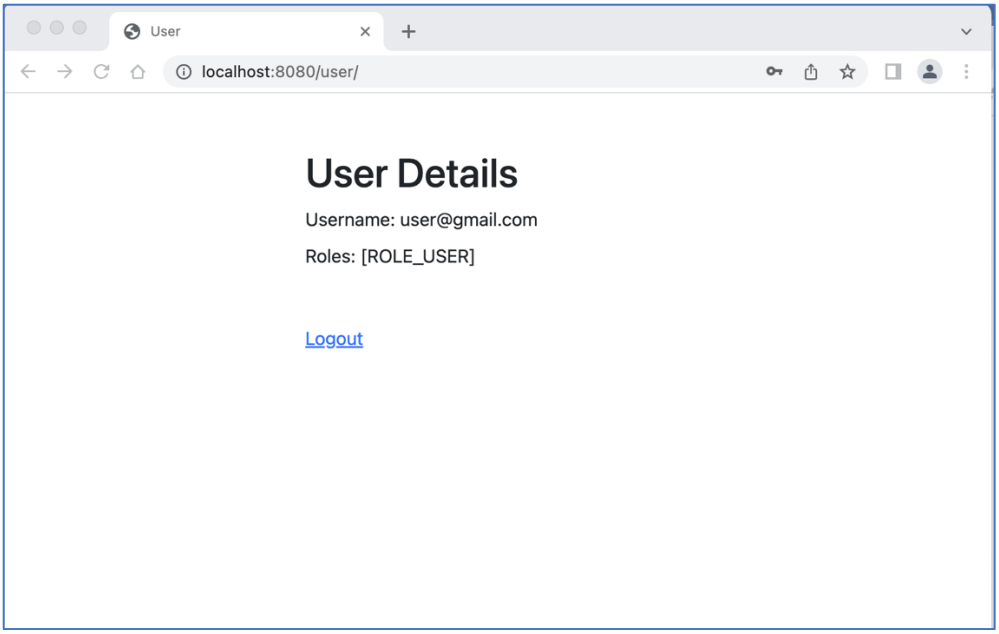
2) Login

Now that a new user has been successfully registered, we can use the credentials to login at: http://localhost:8080/login.



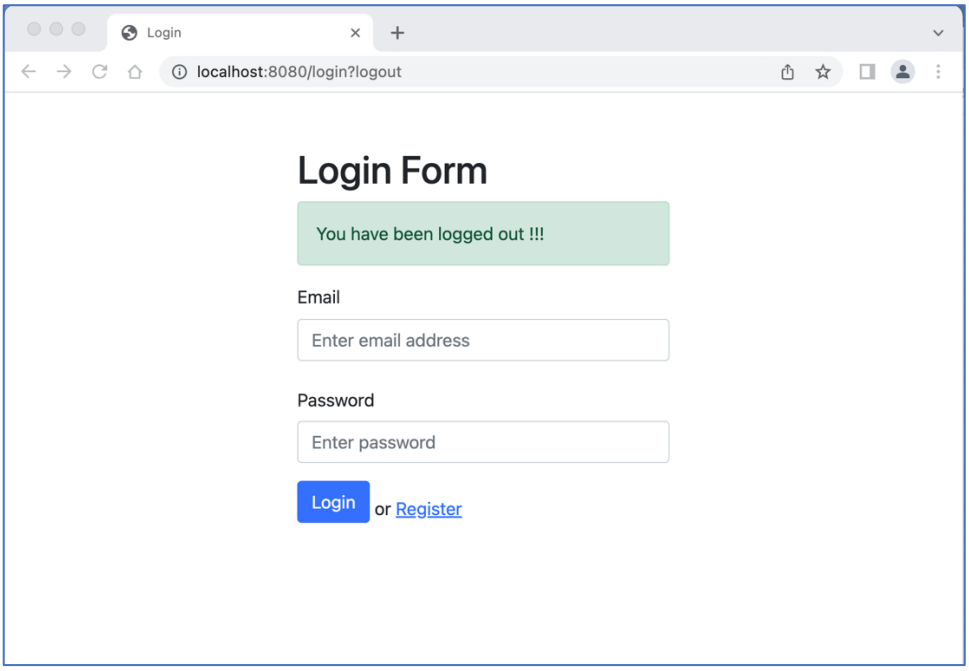
3) Details

The user is now "logged-in" to the system with "role_user" and hence URLs accessible by "role_user" can be viewed: http://localhost:8080/user/

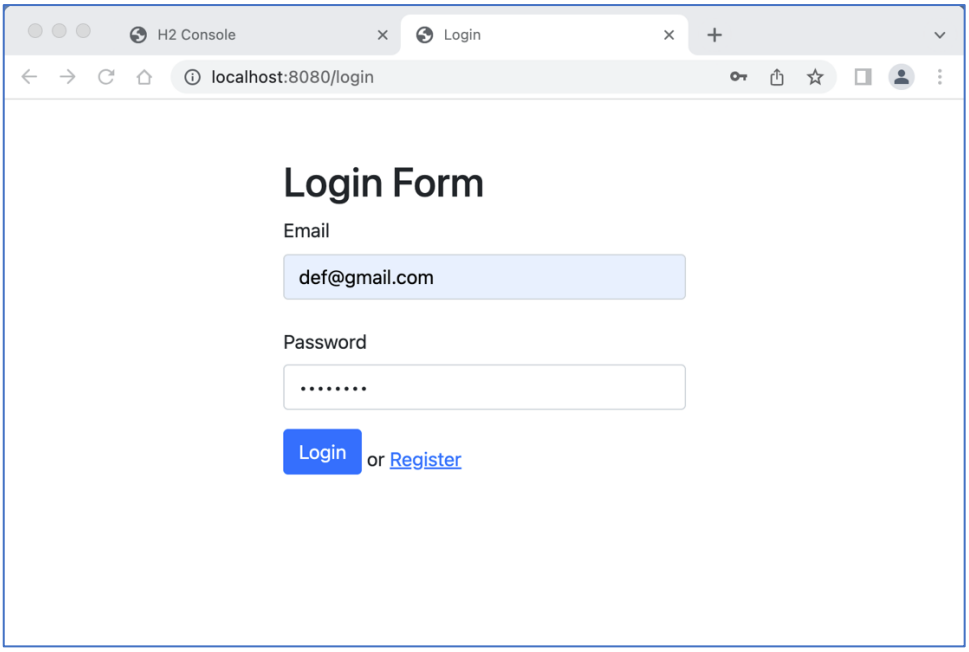


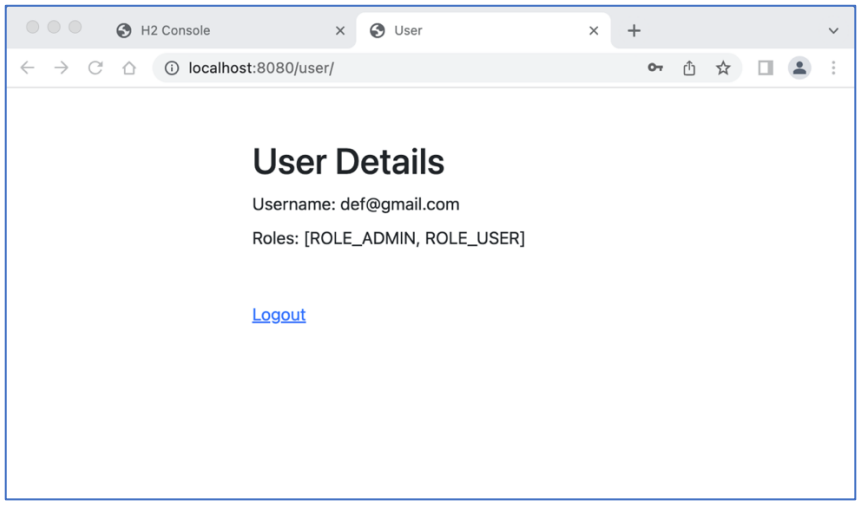
3) Logout

In order to log-out from the system, we can send a get call to http://localhost:8080/logout or click on **Logout** link , please refer to the diagram above.

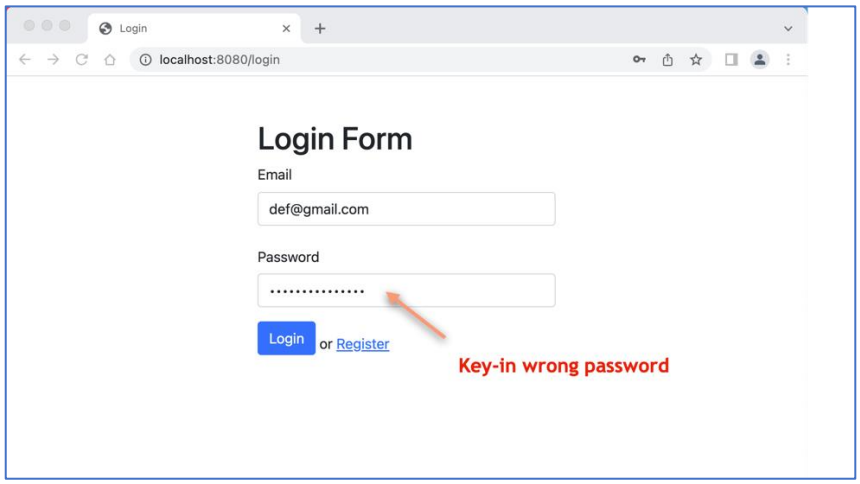


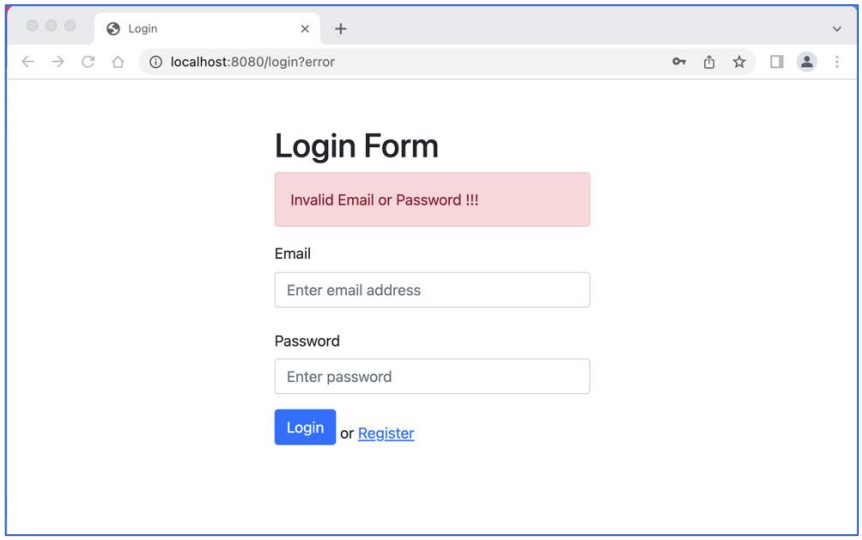
4) Login user with two roles – ADMIN & USER



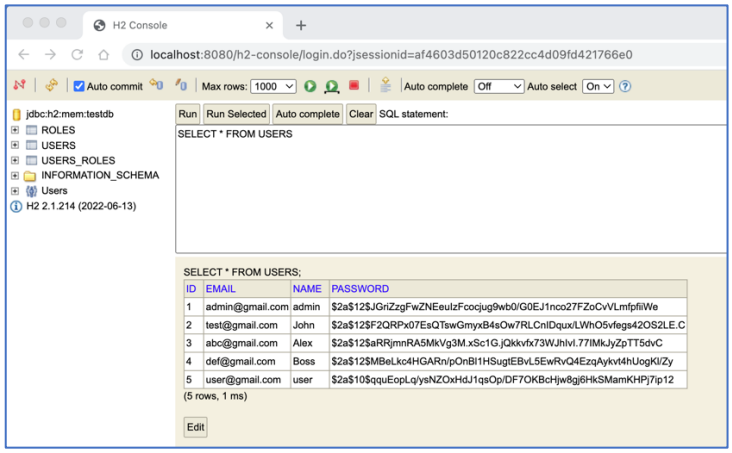


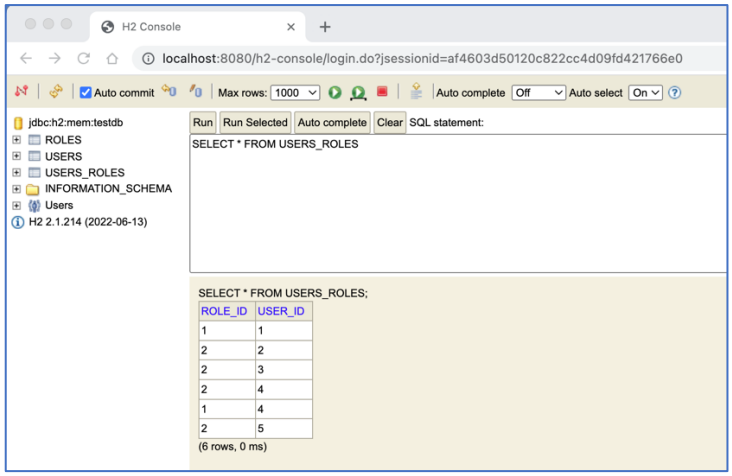
5) Wrong password





H2 Data





Note: USER_ID : 4 having two roles. ROLE_ID: 2 and 1